

Lecture 11: Sorting Foundations

Topics

- Ordering and comparison: what does "sorted" mean?
- Stability: preserving relative order of equal elements
- The comparison-based lower bound: why $O(n \log n)$ is optimal
- Multi-key sorting: sorting by multiple criteria
- Custom comparators and key functions

Goals

- Understand Python's `sorted()` and `.sort()` thoroughly
- Recognize when stability matters (and when it doesn't)
- Prove why comparison sorts can't beat $O(n \log n)$
- Sort tracks by artist, then title, then popularity
- Use `functools.cmp_to_key` for complex orderings

Roadmap

First half (≈45 min)

- What is sorting? Ordering relations and comparisons
- Python's sorting: `sorted()` vs `.sort()`, key functions
- Stability: definition, examples, why it matters
- Multi-key sorting with tuples
- In-class exercise 1 (commit required)

Break (3 min)

Second half (≈45 min)

- The decision tree model of comparison-based sorting
- Lower bound proof: why $O(n \log n)$ is optimal
- Custom comparators with `functools.cmp_to_key`
- Sorting algorithms preview (next lecture)
- In-class exercise 2 (commit required)
- Complexity checkpoints and wrap-up

Setup: load Spotify data

We'll use tracks throughout this lecture.

```
In [ ]: import csv
        from functools import cmp_to_key
        from collections import Counter

        # Load tracks
        tracks = {}
        with open('data/tracks.csv') as f:
            reader = csv.DictReader(f)
            for row in reader:
                tracks[row['track_id']] = row

        # Load plays
        plays = []
        with open('data/plays.csv') as f:
            reader = csv.DictReader(f)
            for row in reader:
                plays.append(row)

        print(f"Loaded {len(tracks)} tracks and {len(plays)} plays")
```

Part 1: What is Sorting?

Sorting: Rearranging elements so they satisfy an **ordering relation**.

For numbers: Natural ordering ($\leq$)

```
[5, 2, 8, 1, 9] → [1, 2, 5, 8, 9]
```

For strings: Lexicographic (dictionary) ordering

```
["zebra", "apple", "banana"] → ["apple", "banana", "zebra"]
```

For tracks: Define your own ordering!

- By artist name (A-Z)
- By play count (most played first)
- By duration (shortest first)
- By release date (newest first)

Key question: What does "less than" mean for your data?

Total ordering: the three properties

For sorting to work, the ordering relation $\leq$ must be **total**:

1. Reflexive

Every element is equal to itself: $a \leq a$

2. Antisymmetric

If $a \leq b$ and $b \leq a$, then $a = b$

3. Transitive

If $a \leq b$ and $b \leq c$, then $a \leq c$

Why this matters: If your comparison function violates these properties, sorting produces undefined behavior!

Bad example: "Rock beats scissors, scissors beats paper, paper beats rock" $\rightarrow$ not transitive $\rightarrow$ can't sort!

Python's sorting: sorted() vs .sort()

Two ways to sort in Python:

1. `sorted()` — returns a new sorted list

```
numbers = [5, 2, 8, 1, 9]
sorted_nums = sorted(numbers) # Returns new list
# numbers is unchanged: [5, 2, 8, 1, 9]
# sorted_nums: [1, 2, 5, 8, 9]
```

2. `.sort()` — sorts in-place

```
numbers = [5, 2, 8, 1, 9]
numbers.sort() # Modifies numbers in place
# numbers: [1, 2, 5, 8, 9]
# Returns None
```

When to use each:

- Use `sorted()` when you need the original list unchanged
- Use `.sort()` when you want to save memory (no copy)

Key functions: defining custom orderings

```
In [ ]: # Sort tracks by artist name
track_list = list(tracks.values())[:10]

# By artist name (A-Z)
by_artist = sorted(track_list, key=lambda t: t['artist'])
print("\nSorted by artist:")
for t in by_artist[:5]:
    print(f" {t['artist']}: {t['name']}")

# By track name (A-Z)
by_name = sorted(track_list, key=lambda t: t['name'])
print("\nSorted by track name:")
for t in by_name[:5]:
    print(f" {t['name']} by {t['artist']}")

# By duration (shortest first)
by_duration = sorted(track_list, key=lambda t: int(t['duration_ms']))
print("\nSorted by duration:")
for t in by_duration[:5]:
    duration_sec = int(t['duration_ms']) / 1000
    print(f" {duration_sec:.1f}s: {t['name']}")
```

Key functions: how they work

The **key** parameter:

```
sorted(items, key=func)
```

Process:

1. For each item `x`, compute `key_value = func(x)`
2. Sort items by comparing their `key_value`s
3. Return items in sorted order

Example:

```
words = ["apple", "pie", "banana", "zoo"]
sorted(words, key=len)
# Computes: [("apple", 5), ("pie", 3), ("banana", 6), ("zoo", 3)]
# Sorts by: 3, 3, 5, 6
# Returns: ["pie", "zoo", "apple", "banana"]
```

Key is called once per item (not once per comparison). Python caches the results.

Reverse ordering

```
In [ ]: # Count plays per track
play_counts = Counter(play['track_id'] for play in plays)

# Sort tracks by play count (most played first)
tracks_by_plays = sorted(
    track_list,
    key=lambda t: play_counts.get(t['track_id'], 0),
    reverse=True # Descending order
)

print("Most played tracks:")
for t in tracks_by_plays[:5]:
    count = play_counts.get(t['track_id'], 0)
    print(f" {count:3d} plays: {t['name']} by {t['artist']}")
```

Stability: Preserving Relative Order

Definition: A sorting algorithm is **stable** if equal elements maintain their relative order.

Example:

```
Original:  [("Alice", 5), ("Bob", 3), ("Charlie", 5), ("David", 3)]
Sort by score:

Stable:    [("Bob", 3), ("David", 3), ("Alice", 5), ("Charlie", 5)]
           ↑ Bob before David (both have 3)
           ↑ Alice before Charlie (both have 5)

Unstable:  [("David", 3), ("Bob", 3), ("Charlie", 5), ("Alice", 5)]
           ↑ David before Bob (order flipped!)
```

Python's `sorted()` and `.sort()` are stable (use Timsort).

Why stability matters

```
In [ ]: # Tracks with same artist
sample_tracks = [
    {'artist': 'Beatles', 'name': 'Hey Jude', 'year': '1968'},
    {'artist': 'Beatles', 'name': 'Let It Be', 'year': '1970'},
    {'artist': 'Beatles', 'name': 'Come Together', 'year': '1969'},
    {'artist': 'Stones', 'name': 'Paint It Black', 'year': '1966'},
    {'artist': 'Stones', 'name': 'Satisfaction', 'year': '1965'},
]

# Sort by artist (stable sort)
by_artist = sorted(sample_tracks, key=lambda t: t['artist'])

print("Sorted by artist (stable):")
for t in by_artist:
    print(f" {t['artist']}: {t['name']} ({t['year']})")

print("\nNotice: Beatles tracks maintain chronological order!")
print("This is because the sort is stable.")
```

Multi-key sorting with tuples

Problem: Sort tracks by artist (A-Z), then by name (A-Z) within each artist.

Solution: Use tuples as keys!

```
sorted(tracks, key=lambda t: (t['artist'], t['name']))
```

How it works: Python compares tuples lexicographically:

1. Compare first elements
2. If equal, compare second elements
3. If equal, compare third elements
4. ...

```
("Alice", 100) < ("Bob", 50)      → True   (Alice < Bob)
("Alice", 100) < ("Alice", 200)   → True   (100 < 200)
("Alice", 100) < ("Alice", 100)   → False  (equal)
```

Multi-key sorting example

```
In [ ]: # Sort by artist, then by track name
by_artist_then_name = sorted(
    sample_tracks,
    key=lambda t: (t['artist'], t['name'])
)

print("Sorted by artist, then name:")
for t in by_artist_then_name:
    print(f" {t['artist']}: {t['name']} ({t['year']})")

# Sort by artist, then by year (newest first within each artist)
by_artist_then_year = sorted(
    sample_tracks,
    key=lambda t: (t['artist'], -int(t['year'])) # Negate year for reverse
)

print("\nSorted by artist, then year (newest first):")
for t in by_artist_then_year:
    print(f" {t['artist']}: {t['name']} ({t['year']})")
```

Multi-key sorting: mixing ascending/descending

Problem: Sort by play count (descending), then by artist name (ascending).

Challenge: Can't use `reverse=True` for both!

Solution 1: Negate numeric keys

```
key=lambda t: (-play_count, artist_name)
```

Solution 2: Multiple stable sorts (reverse order)

```
# Sort by secondary key first  
tracks.sort(key=lambda t: t['artist'])  
# Then by primary key (stable sort preserves secondary)  
tracks.sort(key=lambda t: play_counts[t['track_id']], reverse=True)
```

Which to use: Solution 1 is clearer and more efficient (one pass).

Exercise 1: Multi-key sorting

Task: Sort tracks by multiple criteria.

Requirements:

1. Take the first 20 tracks
2. Sort by:
 - Artist name (A-Z)
 - Duration (shortest first) within each artist
 - Track name (A-Z) if duration is equal
3. Print the sorted list showing artist, duration, and track name

Hint: Use a 3-tuple as the key:

```
key=lambda t: (t['artist'], int(t['duration_ms']), t['name'])
```

Commit required: Commit your solution before the break.

```
In [ ]: # YOUR CODE HERE  
# Sort tracks by artist, then duration, then name
```

Break (3 minutes)

Commit your Exercise 1 solution now!

When we return:

- The comparison-based lower bound proof
- Custom comparators with `functools.cmp_to_key`
- Preview of sorting algorithms

Part 2: The Comparison-Based Lower Bound

Big question: Can we sort faster than $O(n \log n)$?

Answer: Not with comparison-based sorting!

Comparison-based: Algorithm only learns about order by comparing pairs of elements.

- $a < b?$ → Yes or No
- Cannot inspect values directly (like digits in radix sort)

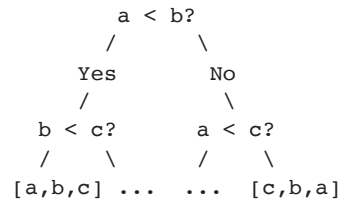
Examples:

- Comparison-based: Merge sort, quicksort, heapsort, insertion sort
- Not comparison-based: Counting sort, radix sort, bucket sort

Theorem: Any comparison-based sorting algorithm must make at least $\Omega(n \log n)$ **comparisons** in the worst case.

Decision tree model

Model: Every comparison-based sorting algorithm can be represented as a **decision tree**.



- **Internal nodes:** Comparisons ($a < b?$)
- **Leaves:** Permutations (sorted orders)
- **Path from root to leaf:** Sequence of comparisons to sort
- **Height of tree:** Worst-case number of comparisons

Lower bound proof (intuition)

Key insight: The decision tree must have at least **$n!$ leaves**.

Why? There are $n!$ possible permutations of n elements. Each permutation must be a distinct leaf (or the algorithm wouldn't work correctly for all inputs).

Binary tree with $n!$ leaves has height at least $\log_2(n!)$

Using Stirling's approximation:

$$\begin{aligned}\log_2(n!) &= \log_2(1) + \log_2(2) + \dots + \log_2(n) \\ &\approx n \log_2(n) - n \log_2(e) \\ &= \Theta(n \log n)\end{aligned}$$

Conclusion: Any comparison-based sort requires **$\Omega(n \log n)$ comparisons** in the worst case.

This is a lower bound — you can't do better with comparisons alone!

Why the lower bound matters

Practical implications:

1. Merge sort, heapsort, and Timsort are asymptotically optimal

- All achieve $O(n \log n)$ worst case
- Can't do fundamentally better with comparisons

2. Quicksort is optimal on average

- $O(n \log n)$ average case
- $O(n^2)$ worst case, but rare with good pivot selection

3. To beat $O(n \log n)$, you need non-comparison sorts

- Counting sort: $O(n + k)$ where k is range of values
- Radix sort: $O(nk)$ where k is number of digits
- But these require special properties of the data

4. Python's `sorted()` is already optimal

- Uses Timsort ($O(n \log n)$ worst case, stable)
- Don't try to "optimize" by writing your own sort!

Custom comparators with `functools.cmp_to_key`

Problem: Sometimes you need a comparison function, not a key function.

Example: Sort tracks by "popularity" (a complex formula).

Old Python 2 style:

```
def compare(a, b):  
    if a < b: return -1  
    if a > b: return 1  
    return 0
```

Python 3: Use `functools.cmp_to_key` to convert:

```
from functools import cmp_to_key  
  
sorted(items, key=cmp_to_key(compare_func))
```

Custom comparator example

```
In [ ]: def compare_tracks_by_popularity(t1, t2):  
        """Custom comparison: sort by play count, break ties with recency."""  
        count1 = play_counts.get(t1['track_id'], 0)  
        count2 = play_counts.get(t2['track_id'], 0)  
  
        # Primary: more plays is better (reverse)  
        if count1 != count2:  
            return count2 - count1 # Descending  
  
        # Tie-break: shorter tracks first  
        dur1 = int(t1['duration_ms'])  
        dur2 = int(t2['duration_ms'])  
        if dur1 != dur2:  
            return dur1 - dur2 # Ascending  
  
        # Final tie-break: alphabetical by name  
        if t1['name'] < t2['name']:  
            return -1  
        elif t1['name'] > t2['name']:  
            return 1  
        return 0  
  
        # Use custom comparator  
        sorted_tracks = sorted(  
            track_list,  
            key=cmp_to_key(compare_tracks_by_popularity)
```

)

```
print("Sorted by custom popularity:")
for t in sorted_tracks[:5]:
    count = play_counts.get(t['track_id'], 0)
    duration = int(t['duration_ms']) / 1000
    print(f"  {count:3d} plays, {duration:.0f}s: {t['name']}")
```

When to use cmp_to_key

Prefer key functions when possible:

- Simpler and more efficient
- Key function called once per item
- Comparison function called $O(n \log n)$ times

Use cmp_to_key when:

1. Comparison logic is too complex for a key function
2. You need to compare two items directly (relative comparison)
3. You're porting old Python 2 code that used `cmp` parameter

Most of the time, tuple keys are enough:

```
# Instead of complex comparator:  
key=lambda t: (-play_counts[t['track_id']], int(t['duration_ms']),  
t['name'])
```


Common pitfalls

1. Modifying list while sorting

```
#  Bad: don't modify during iteration
for track in sorted_tracks:
    if condition:
        sorted_tracks.remove(track) # Modifies during iteration!
```

2. Forgetting that .sort() returns None

```
#  Bad: .sort() returns None
sorted_tracks = tracks.sort(key=lambda t: t['name'])
# sorted_tracks is None!

#  Good:
tracks.sort(key=lambda t: t['name'])
# Now tracks is sorted in-place
```

3. Non-transitive comparisons

```
#  Bad: violates transitivity
def bad_compare(a, b):
    return random.choice([-1, 0, 1]) # Undefined behavior!
```

Sorting algorithms preview

Next lecture: We'll explore the algorithms behind `sorted()`.

Algorithm	Best	Average	Worst	Space	Stable
Insertion sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes
Merge sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	Yes
Quicksort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$	No
Heapsort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$	No
Timsort	$O(n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	Yes

Python uses Timsort:

- Hybrid of merge sort and insertion sort
- Exploits existing order in data (adaptive)
- Stable, $O(n \log n)$ worst case
- Best real-world performance

Exercise 2: Sorting quiz

Answer these questions (write in a comment):

1. What is the minimum number of comparisons needed to sort 100 elements using a comparison-based algorithm?

- A) 100
- B) 1000
- C) ~ 664 ($100 \log_2 100$)
- D) 10,000

2. Why does stability matter when sorting by artist then by year?

- A) It doesn't matter
- B) It preserves the year order within each artist
- C) It makes the sort faster
- D) It reduces memory usage

3. Which is more efficient for a single sort?

- A) `key=lambda t: (t['artist'], t['name'])`
- B) Two stable sorts (first by name, then by artist)

Commit required: Commit your answers before class ends.

```
In [ ]: # YOUR ANSWERS HERE (as comments)
```

```
# 1. Answer:
```

```
# 2. Answer:
```

```
# 3. Answer:
```

Complexity checkpoints

Question 1: What is the time complexity of `sorted(tracks, key=lambda t: t['name'])` ?

- A) $O(n)$ — just one pass through tracks
- B) $O(n \log n)$ — Python uses Timsort
- C) $O(n^2)$ — nested comparisons
- D) $O(\log n)$ — binary search

Question 2: Why can't we sort n elements faster than $O(n \log n)$ using comparisons?

- A) All known algorithms are $O(n \log n)$
- B) Decision tree must have $n!$ leaves, height is $\Omega(n \log n)$
- C) Comparisons are expensive
- D) Python is slow

Question 3: Is Python's `sorted()` stable?

- A) No
- B) Yes, it uses Timsort which is stable

Answers: B ($O(n \log n)$), B (lower bound proof), B (stable Timsort)

Key definitions

Sorting: Rearranging elements to satisfy an ordering relation.

Total ordering: A relation that is reflexive, antisymmetric, and transitive.

Stability: Property that equal elements maintain their relative order after sorting.

Comparison-based sort: Algorithm that only learns order by comparing pairs of elements.

Lower bound: Minimum complexity required by any algorithm in a class ($\Omega(n \log n)$ for comparison sorts).

Key function: Function that extracts a comparison key from each element.

Timsort: Python's sorting algorithm (hybrid of merge sort and insertion sort, stable, $O(n \log n)$ worst case).

Wrap-up: Sorting foundations

You've learned:

-  `sorted()` vs `.sort()`: when to use each
-  Key functions define custom orderings
-  Stability preserves relative order of equal elements
-  Multi-key sorting with tuples: `(artist, name, year)`
-  Comparison-based lower bound: $O(n \log n)$ is optimal
-  Decision tree proof: must have $n!$ leaves $\rightarrow \Omega(n \log n)$ height
-  `functools.cmp_to_key` for complex comparisons
-  Python's Timsort is stable and optimal

Next lecture: Heapsort vs quicksort vs mergesort tradeoffs

Don't forget: Commit both exercises before you leave!

